

AtmoVista - Internship Project Report

End-to-end AQI forecasting for Pakistan with MLOps, live deployment, and GitHub Actions automation.

Project title: AtmoVista (Pearls AQI Predictor)

Forecast window: Next 3 days (+24h, +48h, +72h)

Cities: Lahore, Karachi, Islamabad, Peshawar, Quetta

Repository: Hosted on GitHub with GitHub Actions CI/CD

Deployment: API on Render, frontend on Vercel, features/models on Hopsworks

1. Introduction

AtmoVista is an end-to-end machine learning system that predicts Air Quality Index (AQI) for five major Pakistan cities over the next three days. The full pipeline is implemented, pushed to GitHub, automated with GitHub Actions, and deployed for live use.

The system includes:

- automated data collection
- feature engineering and historical backfill
- multi-model training and evaluation
- Hopsworks feature store and model registry
- CI/CD with GitHub Actions (hourly features + daily training)
- interactive AtmoVista web dashboard (FastAPI + React)

The goal is practical: help everyday users understand current and upcoming air quality, while also showing model performance and explainability for technical review.

2. Problem statement

Air quality in Pakistan varies strongly by city and season. A useful system should:

- Forecast AQI for the next 24, 48, and 72 hours
- Update regularly with fresh data
- Explain predictions (not just show a number)
- Alert when conditions become unhealthy or hazardous
- Run on a free / serverless-friendly stack

AtmoVista addresses these needs with an MLOps pipeline and a dual-mode dashboard (For you / For experts).

3. Objectives completed

Objective	Status
Predict AQI for next 3 days	Completed
Feature pipeline (fetch to feature store)	Completed
Historical backfill for training	Completed
Train and evaluate multiple models	Completed
Store models in model registry	Completed
Automate hourly and daily pipelines	Completed and running
Interactive AtmoVista dashboard	Completed

Objective	Status
EDA	Completed
SHAP explainability	Completed
Hazardous AQI alerts	Completed
GitHub + Render + Vercel + Hopsworks	Completed

UI note: The brief mentions Streamlit/Gradio. This project uses FastAPI + React because the mentor allowed a stronger custom frontend for higher UI marks.

4. System overview

AtmoVista follows this flow in production:

- Open-Meteo APIs provide pollutant and weather data
- Feature and backfill pipelines engineer hourly features
- Features are stored in the Hopsworks Feature Store
- Training compares Ridge, Random Forest, HistGradientBoosting, and LSTM
- Winning models are registered in the Hopsworks Model Registry
- FastAPI loads models/features and serves forecasts
- The AtmoVista React dashboard consumes the API
- GitHub Actions keeps features and models updated on schedule

Main layers

Layer	Responsibility
Data	Open-Meteo air quality and weather
Features	Time, lag, rolling, future-weather features and targets
Storage	Hopsworks Feature Group and Model Registry
Training	Per-horizon model selection
Serving	FastAPI inference API
Presentation	AtmoVista React dashboard
Automation	GitHub Actions hourly and daily jobs

5. Technology stack

Area	Choice	Reason
Language	Python	Required for ML pipelines
Classical ML	Scikit-learn	Ridge / RF / HistGB
Deep learning	TensorFlow LSTM	Advanced model requirement
Feature store	Hopsworks	Feature Store + Model Registry

Area	Choice	Reason
Orchestration	GitHub Actions	Free CI/CD alternative to Airflow
API	FastAPI	Backend for serving forecasts
UI	React + Vite + TypeScript	Mentor-approved custom frontend
Explainability	SHAP (local + global)	Per-horizon signed drivers / XAI
Deploy	Render + Vercel	Production hosting (Render Standard for demo)

6. Data collection

Source: Open-Meteo Air Quality API and Weather API (free, no paid key). The brief lists AQICN/OpenWeather as examples and allows exploring other options.

Pollutants / AQI

- PM2.5, PM10, CO, NO2, SO2, O3
- US AQI (preferred) / European AQI fallback

Weather

- temperature, humidity, precipitation
- wind speed / direction
- cloud cover, surface pressure

Cities and coordinates

City	Latitude	Longitude
Lahore	31.5204	74.3587
Karachi	24.8607	67.0011
Islamabad	33.6844	73.0479
Peshawar	34.0151	71.5249
Quetta	30.1798	66.9750

Training scale

Item	Value
Backfill window	365 days
Dataset size (EDA)	43,800 hourly rows
Cities covered	5

7. Feature engineering

Implemented in src/features/engineering.py.

Group	Examples
Raw signals	pollutants, weather, current AQI
Time features	hour, day, day_of_week, month, sin/cos encodings
Derived features	AQI change rate, temp x humidity, wind x PM2.5
Lag features	AQI and PM2.5 lags at 1, 3, 6, 12, 24, 48, 72 hours
Rolling stats	mean/std over 6h, 12h, 24h windows
Future weather	weather at t+24 / t+48 / t+72
Targets	aqi_target_24h, aqi_target_48h, aqi_target_72h

Design choices:

- Per-horizon models instead of one multi-output model (better accuracy)
- Realistic mode adds controlled noise to future-weather features during training
- Prediction intervals (80%) estimated from validation residuals

8. Exploratory data analysis (EDA)

EDA notebook: notebooks/eda.ipynb. Plots include timeseries, correlation heatmap, seasonality, and city boxplots.

Key findings:

- Lahore has the highest median AQI and widest spread (strong smog spikes).
- Karachi is the most stable city with the lowest mean AQI in this dataset.
- PM2.5 and PM10 are the strongest linear correlates of AQI.
- Evening/night hours and winter months (Nov to Jan) tend toward higher mean AQI.
- City-specific behavior justifies shared features but different error profiles.

City AQI summary (from EDA)

City	Mean	Std	Median	Max
Islamabad	113.0	30.7	107.0	206
Karachi	90.1	21.8	85.0	165
Lahore	153.0	49.1	154.0	364
Peshawar	114.6	29.0	111.0	212
Quetta	86.4	31.7	77.0	332

Forecast difficulty by city

Difficulty	Cities
Hardest	Lahore, Quetta
More stable	Karachi, Peshawar

9. Training pipeline

Script: pipelines/training_pipeline.py

Models experimented

Model	Library
Ridge Regression	Scikit-learn
Random Forest	Scikit-learn
HistGradientBoosting	Scikit-learn
LSTM	TensorFlow

Evaluation metrics

Metric	Purpose
RMSE	Main selection metric
MAE	Average absolute error
R2	Explained variance
Category accuracy	AQI band match for user-facing quality

Best model is selected per horizon using validation RMSE.

Horizon	Selected model
+24h	Ridge
+48h	HistGradientBoosting
+72h	Ridge

Winner label: ridge@24h+hist_gradient_boosting@48h+ridge@72h | Trained: 2026-08-11

10. Results

10.1 Validation metrics (winner models)

Horizon	Winner	RMSE	MAE	R2	Category Acc.
+24h	Ridge	21.63	14.33	0.716	72.8%
+48h	HistGB	26.22	18.64	0.583	59.9%
+72h	Ridge	28.47	19.78	0.508	61.2%

10.2 Test metrics (winner models)

Horizon	Winner	RMSE	MAE	R2	Category Acc.
+24h	Ridge	20.63	14.77	0.611	67.8%

Horizon	Winner	RMSE	MAE	R2	Category Acc.
+48h	HistGB	26.63	19.26	0.352	58.1%
+72h	Ridge	28.18	20.90	0.275	54.9%

10.3 Interpretation

- Short-horizon (+24h) forecasts are clearly strongest.
- Error grows with horizon, which is expected for air-quality forecasting.
- Models beat a simple persistence baseline on the tracked horizons.
- Category accuracy matters because users act on bands, not only exact AQI.

10.4 Live monitoring (forecast vs actual)

Source: artifacts/monitoring/summary.json

Scope	MAE	RMSE	R2	Category Acc.	Scored rows
Overall	12.33	19.32	0.744	74.8%	3500
+24h	10.42	15.84	0.830	78.8%	1170
+48h	13.48	20.56	0.709	72.1%	1165
+72h	13.10	21.13	0.691	73.4%	1165

Per-city live MAE

City	MAE	Category Acc.	Scored rows
Karachi	4.68	97.6%	700
Islamabad	13.15	66.7%	700
Peshawar	12.95	70.1%	700
Quetta	14.51	71.1%	700
Lahore	16.36	68.3%	700

Karachi performs best in live monitoring. Lahore is the hardest city among the five.

11. Feature store and model registry (Hopsworks)

Item	Value
Feature Group	aqi_features (v3)
Feature contents	Engineered hourly features + targets for 5 cities
Model Registry	aqi_forecast_model
Bundle	Per-horizon winners synced by hopsworks_sync.py

Production mode (STORAGE_MODE=hopsworks) loads features from Hopsworks and keeps an in-memory cache in the API. The deployed app uses Hopsworks as the source of truth.

12. Inference and API

API module: api/main.py

Endpoint area	What it serves
Forecast	Current AQI + 3-day forecast
History	Recent AQI history
Weather	Temperature, humidity, wind, pollutant driver
Health tips / exercise	Practical guidance
Alerts	Hazardous / unhealthy warnings
Explain	Per-horizon SHAP, waterfall, narrative, city compare
Leaderboard / ops / monitoring	Model and pipeline health

Prediction flow:

- Load latest feature row for selected city
- Fill future-weather fields from Open-Meteo forecast
- Run the horizon-specific winner model
- Apply prediction interval
- Map AQI to EPA-style category + color
- Return JSON to the AtmoVista frontend

13. AtmoVista dashboard (frontend)

Built with React + Vite + TypeScript and deployed on Vercel.

For you (everyday mode)

- Pakistan live map + city rankings
- Current AQI gauge with category bands
- Weather strip, pollutant breakdown, 3-day outlook with calendar dates
- Confidence / peak / improvement insights
- Health tips, alerts, exercise advice, smog season panel, city watchlist

For experts (technical mode)

- Ops strip (storage mode, model, sync)
- Beat-the-baseline panel and pipeline health board
- Model leaderboard
- XAI: per-horizon SHAP, signed local drivers, waterfall, narrative
- Global drivers, city SHAP compare, glossary hover, pollutant link
- Live accuracy table (forecast vs actual)

14. Alerts and health guidance

Feature	Behavior
Alerts	Shown for unhealthy/hazardous current or forecast AQI with date
Health tips	Category-based guidance for masking, exposure, sensitive groups

Feature	Behavior
Exercise advice	Decision card for outdoor exercise based on near-term AQI

15. Explainability (SHAP)

Implemented in `src/inference/explain.py` and shown in the expert dashboard (`/aqi/explain`, plus `compare/global` helpers).

View	Purpose
Per-horizon tabs	Separate explanations for +24h, +48h, and +72h winners
City-level importance	Mean SHAP over recent city rows
Signed local explanation	Latest-hour SHAP with direction (raises / lowers AQI)
Waterfall	Step path from baseline value to predicted AQI
Plain-language narrative	Short sentence summarizing the top drivers
Global training drivers	Surfaces artifacts/shap_summary.json from training
City compare	Top SHAP drivers across Pakistan cities
Feature glossary	Hover tooltips for lag / weather / pollutant names
Pollutant link	Whether live chemistry and SHAP agree on signal family

Typical important drivers include recent AQI/PM lags, rolling AQI statistics, and weather-linked features.

16. Automation / CI-CD (GitHub Actions)

The repository is on GitHub. Scheduled workflows keep the system updated.

Workflow	Schedule	What it does
feature-hourly.yml	Hourly	Refresh features + reconcile monitoring
training-daily.yml	Daily	Retrain models + sync to Hopsworks
monitor-daily.yml	Daily	Maintain scored forecast-vs-actual history

Repository secrets used by Actions: `HOPSWORKS_API_KEY`, `HOPSWORKS_PROJECT`, `HOPSWORKS_HOST` (optional).

17. Deployment

Service	Platform	Status
AtmoVista frontend	Vercel	Deployed
FastAPI backend	Render (Standard for reliable demo)	Deployed
Feature store / models	Hopsworks	Live source of truth

Production configuration:

- Render uses STORAGE_MODE=hopsworks and Hopsworks credentials
- Vercel uses VITE_API_URL pointing to the Render API
- CORS allows the Vercel frontend origin

18. Creative / unique additions

Addition	Why it helps
5 Pakistan cities	Broader coverage than a single-city demo
Dual audience UI	Everyday users and technical reviewers
Live weather + pollutant breakdown	Context around the AQI number
Richer XAI (signed SHAP, waterfall, narrative)	Clearer trust for technical reviewers
Prediction confidence bands	Uncertainty communication
Peak / improvement insights	Actionable short-term planning
Smog season calendar	Seasonal awareness
Exercise recommendation	Practical health decision
Live monitoring board	Forecast vs actual trust check
Beat-persistence baseline	Shows model value clearly
City watchlist	Personalization
Calendar dates on horizons	Clearer forecast timing

19. Limitations

- Open-Meteo AQI is model-based and may differ from local ground stations.
- Longer horizons (+48h / +72h) have higher error by nature.
- Hopsworks free-plan usage limits, and hosting plan choices (Render/Vercel), can still affect latency and cost if left on always-on Standard longer than needed.
- LSTM was trained and compared, but classical models currently win most horizons.

20. Future work

- Fuse station-level AQI (where available) for calibration
- Probabilistic category forecasts (not only point AQI)
- City-specific specialist models
- Push notifications for hazardous alerts
- Longer live monitoring window for stronger trust metrics
- Stronger uptime monitoring for the public demo

21. Conclusion

AtmoVista completes the internship requirements for an end-to-end AQI prediction system: data ingestion, feature store, multi-model training, registry, automation, explainability, alerts, and an interactive dashboard. The project is implemented, pushed to GitHub, automated with GitHub Actions, and deployed with Render + Vercel +

Hopsworks.

Key numbers:

- Strongest forecasts at +24h (validation R2 about 0.72, category accuracy about 73%)
- Live monitoring over 3500 scored rows: overall MAE about 12.3, category accuracy about 75%

These results support practical short-term planning and health guidance.

22. Appendix - important files

Path	Role
pipelines/feature_pipeline.py	Hourly feature refresh
pipelines/backfill.py	Historical training data
pipelines/training_pipeline.py	Model training and selection
pipelines/hopsworks_sync.py	Feature and model sync
pipelines/monitor_pipeline.py	Forecast vs actual logging
api/main.py	FastAPI serving
frontend/src/App.tsx	AtmoVista dashboard
artifacts/model_leaderboard.json	Training results
notebooks/eda.ipynb	EDA
.github/workflows/	CI/CD automation
docs/AtmoVista_Internship_Report.pdf	This report

Report prepared for internship submission - AtmoVista / Pearls AQI Predictor.